High-Concurrency Distributed Ledger Service

Anarghya Das
Requirements, Software System
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1096@iitj.ac.in

Arnav Vivek
Stakeholders, Team
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1014@iitj.ac.in

Neeli Satwik
Work, Opportunity
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1048@iitj.ac.in

Subhanshu Gupta
Software System, Way of Working
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1072@iitj.ac.in

I. Problem Statement

A. Background

The rapid growth of digital payment platforms, online banking systems, and financial technology applications has significantly increased the demand for high-throughput transaction processing systems. These systems must handle large volumes of concurrent monetary transfers while guaranteeing strong correctness properties such as atomicity, consistency, isolation, and durability. Even a single inconsistency, such as a partial transfer or an incorrect balance update, can lead to financial loss and loss of trust.

Traditional centralized database systems provide strong ACID guarantees but suffer from limited scalability. As the number of concurrent users increases, vertical scaling becomes insufficient due to hardware limits on CPU, memory, disk throughput, and network bandwidth. Consequently, modern transaction systems adopt distributed architectures in which data is partitioned across multiple nodes to enable horizontal scaling. However, distributing data introduces significant complexity in maintaining consistency across shards, especially when transactions span multiple partitions.

B. Core Challenges

Designing a distributed ledger service that scales horizontally while preserving strong transactional guarantees presents several fundamental challenges:

1) **Atomic Cross-Shard Execution:** When a transaction involves accounts stored on different shards, the system must ensure that either all updates commit or all abort. Partial commits must never occur.

2) **Durability Under Failures:** The system must guarantee that once a transaction is committed, it survives process crashes, replica failures, and leader changes.

3) **Consistency Preservation:** Financial invariants must be maintained at all times, including:
   - Conservation of total monetary value across the system.
   - Prevention of negative account balances.
   - No duplication or loss of transactions.

4) **High Concurrency:** The system must process large numbers of concurrent transactions without introducing race conditions or violating isolation properties.

5) **Fault Tolerance:** The system must tolerate single-node failures without data loss and must recover automatically through the help of replicas and log replay using WAL.

6) **Scalability Without Global Coordination Bottlenecks:** Adding shards should increase throughput proportionally, without introducing centralized coordination limits.

C. Problem Definition

The problem addressed in this work is the design and implementation of a horizontally scalable distributed ledger service that supports real-time monetary transfers with strong consistency guarantees under concurrent and failure-prone environments.

Formally, the system must satisfy the following requirements:

- Accept transactions consisting of a unique identifier, source account, destination account, and transfer amount.
- Execute single-shard transactions with low latency and minimal coordination overhead.
- Execute cross-shard transactions atomically using two phase commit protocols.
- Persist all write operations using a write-ahead logging mechanism prior to state modification.
- Replicate shard data across multiple replicas and commit writes only after majority acknowledgment.
- Automatically detect and recover from database failures.
- Preserve correctness invariants even during crash recovery and reprocessing.

The system must operate on commodity hardware within a single datacenter environment, assume fail-stop node failures, and rely on crash-recovery storage semantics. Byzantine behavior and geo-distributed consistency guarantees are explicitly out of scope.

D. Objectives and Target Metrics

Beyond functional correctness, the system aims to achieve the following measurable performance objectives:

- Support a minimum throughput of 100 transactions per second per shard.
- Maintain single-shard transaction latency below 50 ms at the 99th percentile.
- Maintain cross-shard transaction latency below 200 ms at the 99th percentile.
- Scale throughput horizontally with the addition of new shards.

These targets reflect a balance between strong consistency guarantees and practical performance expectations for a datacenter-scoped distributed ledger.

E. Motivation

While large-scale distributed databases such as FoundationDB and CockroachDB provide sophisticated global consistency mechanisms, their architectures rely on complex consensus protocols, tightly synchronized clocks, and extensive infrastructure. There is significant educational value in designing a simplified yet realistic distributed ledger that exposes the underlying principles of sharding, distributed coordination, replication, logging, and failure recovery.

This project therefore seeks to construct a high-concurrency ledger service that captures the essential mechanisms of modern distributed transaction engines while maintaining architectural clarity and traceability from requirements to implementation.

II. Review of Existing Technologies and Solutions

Designing a high-concurrency distributed ledger requires balancing scalability, consistency, durability, and operational simplicity. Modern distributed databases address these concerns through sharding, replication, consensus protocols, and multi-version concurrency control. This section evaluates two prominent distributed database systems, CockroachDB and FoundationDB, and analyzes their architectural strengths and trade-offs in the context of a high-concurrency ledger service.

A. FoundationDB

FoundationDB is a distributed transactional key-value store that provides strict ACID guarantees and automatic data distribution. Its architecture separates transaction processing from storage, internally handling sharding, replication, logging, and recovery.

1) Strengths:

- **Built-in Sharding, Two-Phase Commit, WAL, and Replication:** FoundationDB automatically performs key-range sharding, distributed transaction coordination, write-ahead logging, and consensus-based replication. These mechanisms are transparent to application developers, significantly reducing implementation complexity.
- **High Concurrency Performance:** The system employs Multi-Version Concurrency Control (MVCC) combined with optimistic concurrency control. Non-conflicting transactions execute in parallel while preserving serializable isolation. This design enables high throughput in workloads where contention is moderate.
- **Automatic Hot Range Migration:** FoundationDB continuously monitors key-range load. Overloaded ranges are automatically split and redistributed across storage servers without application-level intervention. This dynamic rebalancing enhances throughput scalability and reduces operational overhead.

2) Limitation:

- **Limited Visibility into Recovery Internals:** Core mechanisms such as WAL management, replication logic, and failover orchestration are abstracted away. While beneficial for operational simplicity, this abstraction reduces transparency and fine-grained control, which may limit experimentation and educational exploration of recovery semantics.

B. CockroachDB

CockroachDB is a distributed SQL database that provides PostgreSQL compatibility over a shared-nothing transactional key-value store. It uses the Raft consensus algorithm for replication and Multi-Version Concurrency Control to ensure serializable isolation.

1) Primary Strengths:

- **Strong ACID Guarantees with Horizontal Scalability:** By combining Raft-based consensus replication with MVCC, CockroachDB ensures strict consistency across distributed nodes. Data is partitioned into ranges, each managed by a Raft group, enabling linear scaling of read and write throughput by adding nodes.
- **High Availability and Fault Tolerance:** Every write operation is replicated to a quorum of nodes before commit, ensuring durability even under node failures. Automatic range rebalancing and replica management provide near-zero Recovery Point Objectives and Recovery Time Objectives.
- **Global Multi-Region Deployment:** The system supports geo-partitioning and row-level data placement, allowing latency optimization and regulatory compliance across regions.

2) Architectural Trade-offs:

- **Consensus Latency Overhead:** Each write operation requires coordination with a Raft quorum. This introduces

additional network round trips and increases commit latency compared to single-node databases. For low-latency financial transfers, this overhead may be non-negligible.

- **Resource Intensity:** Background processes such as log compaction, range rebalancing, and consensus maintenance require substantial CPU and memory resources. Basic fault tolerance requires a minimum of three nodes, increasing infrastructure cost and complexity.
- **Clock Synchronization Sensitivity:** The system depends on reasonably synchronized clocks across nodes. Significant clock skew may degrade performance or cause node instability. While manageable in production environments, this introduces operational considerations.
- **OLAP Limitations:** CockroachDB is optimized for transactional workloads. Large analytical queries involving massive scans or complex joins are less efficient compared to columnar database systems.

C. Comparative Analysis in the Context of a High-Concurrency Ledger

Both FoundationDB and CockroachDB provide mature, production-grade solutions for distributed transactions. However, when evaluated specifically for a high-concurrency ledger service designed for educational exploration and architectural transparency, certain trade-offs emerge:

- FoundationDB abstracts core mechanisms such as logging and failover, limiting visibility into internal durability and recovery processes.
- CockroachDB introduces consensus-induced latency overhead on every write, which may increase commit times for simple monetary transfers.
- Operational complexity and resource requirements are significantly higher than a purpose-built, datacenter-scoped ledger engine.

These observations motivate the development of a custom distributed ledger architecture that retains core distributed systems principles such as sharding, write-ahead logging, majority-based replication, and two-phase commit coordination, while maintaining architectural clarity and measurable performance characteristics. The proposed system prioritizes deterministic partitioning, stateless coordination, and explicit failure recovery mechanisms, enabling fine-grained control and experimental evaluation of distributed transaction behavior.

III. Solution Trajectory

The High-Concurrency Ledger Service is designed as a sharded, log-driven distributed system deployed within a single datacenter environment. The architecture emphasizes separation of concerns between transaction ingestion, coordination, storage, replication, and operational monitoring. This layered structure enables horizontal scalability while preserving strict financial correctness guarantees.

A. Version History Analysis

Our development followed an iterative requirements engineering approach, resulting in eight major revisions of the SRS document over a two-week period (January 20 to Febru- ary 4, 2026). Each iteration addressed specific stakeholder feedback, technical constraints, or design insights.

TABLE I
SRS Version Evolution

Ver	Date	Key Changes
1.0	Jan 20	Initial SRS document
1.1	Jan 21	Added REQ-TX-002 and REQ-TX-003; updated architecture to follow sharding pattern
1.2	Jan 24	Added REQ-REP-003 and REQ-REP-004 (failure detection and primary failover)
1.3	Jan 26	Added write-ahead logging requirements (REQ-DATA-002); documented WAL recovery semantics
2.0	Jan 28	Added Kafka integration (REQ-COORD-001); defined stateless coordinator model
2.1	Jan 30	Added load balancing requirements (REQ-DATA-003); documented shard rebalancing strategy
2.2	Jan 31	Added performance requirements (REQ-PERF-001 to REQ-PERF-004); defined latency targets
3.0	Feb 02	Defined the leader election rules

B. Technology Stack

- React.js for dynamic UI components
- Golang-based backed for it's built in concurrency and reliability.
- JWT-based authentication
- Role-Based access control Middleware
- PostgreSQL for RDBMS
- Redis for session management and caching availability queries +
- Docker and Kubernetes for deployment.

C. Development Methodology

1) Agile Scrum Framework: We adopted 2-week sprints:

Sprint 1-2 (Requirements & Design):

- SRS iterations 1.0 to 1.3
- UML modeling

Sprint 3-4 (Core Functionality):

- User authentication and load balancing
- Transaction request workflow

Sprint 5-6 (Advanced Features):

- Shard Conflict detection and WAL persistence
- Redundant Features i.e. replicas for fault tolerance

Sprint 7-8 (Phase 2 Features):

- Load Monitor to keep track of proposed changes
- Algorithm for shifting the users to maintain shard health

2) Alpha Card Progression: Team members managed distinct Alpha aspects:

- **Requirements** (Anarghya): Scope management, prioritization
- **Software System** (Anarghya, Subhanshu): Architecture decisions, tech stack
- **Work** (Satwik): Task breakdown, backlog grooming
- **Opportunity** (Satwik): The value to be achieved
- **Stakeholders** (Arnav): User interviews, feedback incorporation
- **Way of Working** (Subhanshu): Process adherence, retrospectives
- **Team** (Arnav): Collaboration, communication, conflict resolution

D. Coding and Quality Standards

- **Version Control**: Git with feature branching and pull request reviews
- **Code Style**: ESLint (JavaScript), PEP 8 (Python), Prettier for formatting
- **Testing**: Jest/Mocha for unit tests, target 80% code coverage, Selenium for integration tests
- **Continuous Integration**: GitHub Actions or Jenkins for automated testing on commits
- **Documentation**: JSDoc/Sphinx for code documentation, Swagger for API specification

IV. System Architecture

The system is composed of the following primary components:

- *API Gateway*
- *Event Streaming Layer (Kafka)*
- *Transaction Coordinator*
- *Shard Map Service*
- *Shards (Leaders and Followers)*
- *Shard Load Monitor*

These components interact to form a write-optimized transactional pipeline that decouples client interaction from data persistence and execution.

A. API Gateway

The API Gateway serves as the entry point for client requests. Its responsibilities include:

- Validation of transaction structure and parameters.
- Enforcement of rate limiting and request sanitization.
- The API Gateway acts as a Kafka producer, publishing validated transaction events to a streaming topic for downstream coordination.

B. Event Streaming Layer

All validated transactions are published by the API Gateway to the event streaming layer before execution. This layer serves as the durable ingestion backbone of the system and provides:

- Persistent storage of transaction events.
- Ordered delivery within partitions.
- Replay capability for failure recovery.

Transaction Coordinators operate as stateless consumers within a consumer group. Each coordinator processes assigned partitions and commits offsets only after successful handling of events.

In the event of coordinator failure, partition ownership is reassigned to another coordinator instance, which resumes consumption from the last committed offset. Because shard-level execution is idempotent and durable state changes are recorded in shard-level write-ahead logs, replayed events do not result in duplicate or inconsistent state transitions.

C. Coordination Layer

The coordination layer consists of the **Transaction Coordinator** and the **Shard Map**.

1) Transaction Coordinator: The Transaction Coordinator is responsible for:

- Consuming transaction events from the distributed log.
- Determining the target shard(s) for involved accounts.
- Routing requests to appropriate shard leaders.
- Orchestrating two-phase commit when multiple shards are involved.

The Transaction Coordinator maintains no persistent transaction metadata locally. All durability and recovery guarantees are delegated to the distributed ingestion log and shard-level write-ahead logging mechanisms. As a result, coordinators can fail and restart without requiring state reconstruction, since transaction progress can be safely resumed through log replay and idempotent shard execution.

2) Shard Map: The Shard Map maintains the mapping between logical partitions and physical shard servers. Account identifiers are deterministically mapped to partitions using a hash-based scheme. Logical partitions may be reassigned to different physical shards to support load balancing. Additionally, the shard map will be updated during rebalancing.

This indirection layer enables horizontal scalability while maintaining consistent routing.

D. Data Layer: Shards and Replicas

The data layer is composed of multiple shards (default = 3), each managing a subset of account data. Data within each shard is further divided into smaller logical partitions. For example, a shard may store up to ten partitions, where each partition corresponds to a range of hash values. The shard map maintained by the coordinator tracks partition placement and is updated following rebalancing operations.

Also, maximum number of shards shall be:

$$S_{\max} = 10C$$

where C denotes the number of coordinators.

Each shard follows a primary–follower architecture:

- A single leader handles all write operations.
- Followers replicate state from the leader.

- Majority-based acknowledgment ensures consistency and fault tolerance.

Shards are responsible for validating transactions, maintaining account balances, and enforcing ledger invariants such as non-negative balances and conservation of total funds.

E. Operational Monitoring Layer

The Shard Load Monitor continuously observes shard health and workload metrics, including request rate and system utilization. When sustained imbalances are detected, controlled partition migration may be initiated on a weekly or bi-weekly basis.

Partition migration is coordinated to ensure transactional consistency. Logical partitions are temporarily halted during transfer, remapped upon successful migration, and then resumed.

V. Transaction Processing Model

The ledger supports both single-shard and cross-shard transactions. The execution model preserves strict financial invariants while minimizing coordination overhead whenever possible. Each transaction is defined as `T = (txnID, sourceAccount, destinationAccount, amount)`.

The system enforces the following invariants at all times:

- **No negative account balances:**

$$\text{balance}(account) \geq 0$$

- **Conservation of total monetary value:**

$$\sum \text{balance}_{\text{before}} = \sum \text{balance}_{\text{after}}$$

- **No partial commits.**

Transactions are ingested through the messaging layer and routed by the Transaction Coordinator to the appropriate shard(s). Execution behavior depends on whether the transaction spans one shard or multiple shards.

A. Single-Shard Transaction Execution

A transaction is classified as single-shard if both accounts map to the same shard under deterministic partitioning. In this case, distributed coordination is unnecessary and execution is handled entirely by the shard leader, as shown in Algorithm 1.

Execution Semantics: If validation succeeds, the shard appends an uncommitted record to its Write-Ahead Log (WAL). The entry is replicated to followers and must be acknowledged by a majority before the commit index is advanced.

State changes are applied strictly after durable quorum replication. This guarantees that committed transactions survive crashes and that no ledger update occurs without prior persistence.

B. Cross-Shard Transaction Execution

When source and destination accounts reside on different shards, atomicity must be enforced across independent storage units. The system uses a two-phase commit (2PC) protocol coordinated by the Transaction Coordinator.

Let $S = \{S_1, S_2, \ldots, S_n\}$ denote the participating shards.

Algorithm 1 Single-Shard Transaction Execution

Require: $txnID$, $source$, $destination$, $amount$
Ensure: Atomic and durable ledger update

```
if balance(source) < amount then
    return REJECT
end if
Append (txnID, operation, UNCOMMITTED) to WAL
Replicate WAL entry to followers
Wait for majority ACK
Advance commit index
Apply debit and credit to ledger state
return SUCCESS
```

Algorithm 2 Cross-Shard Coordination (Two-Phase Commit)

Require: $txnID$, participating shards S

```
for all S_i ∈ S do
    Send PREPARE to S_i
end for
if all shards respond PREPARED then
    for all S_i ∈ S do
        Send COMMIT to S_i
    end for
    return SUCCESS
else
    for all S_i that responded PREPARED do
        Send ABORT to S_i
    end for
    return ABORT
end if
```

1) Coordinator Protocol: The coordinator executes the procedure described in Algorithm 2.

2) Shard-Side Prepare Handling: Each shard processes PREPARE messages as shown in Algorithm 3.

Commit Phase: Upon receiving a COMMIT message, each shard:

- Appends a `COMMITTED` record to its WAL,
- Replicates the record to followers,
- Advances its commit index,
- Applies the ledger update.

If ABORT is received, the shard appends a `ABORTED` record and releases any reserved state.

C. Atomicity Guarantee

Atomicity across shards is ensured because:

- No shard applies state updates during PREPARE.
- All shards durably log PREPARED before COMMIT is issued.
- If any shard aborts, all prepared shards receive ABORT.

Thus, the transaction either commits everywhere or nowhere.

Algorithm 3 Shard Prepare Handling

Require: PREPARE($txnID$, operation)

```
if validation fails then
    return ABORT
end if
Append (txnID, PREPARED) to WAL
Replicate to followers
Wait for majority ACK
return PREPARED
```

D. Idempotency and Log Replay

Transaction Coordinators are stateless consumers of the distributed log. In failure scenarios, transaction events may be replayed. To prevent duplicate execution:

- Shards track transaction identifiers.
- Duplicate PREPARE or COMMIT messages are safely ignored.
- State transitions are enforced deterministically via WAL records.

This guarantees correctness under coordinator failure and message replay.

E. Performance Implications

Single-shard transactions require:

- One WAL append,
- One replication round.

Cross-shard transactions require:

- Two WAL appends per shard (PREPARE and COMMIT),
- Two replication rounds,
- Additional coordinator messaging.

Therefore, cross-shard transactions incur higher latency due to coordination and durability overhead.

VI. Replication and Failover Mechanism

To ensure durability and fault tolerance, each shard follows a primary–follower replication architecture. Replication guarantees that committed transactions survive node failures while preserving core financial invariants such as conservation of total balance and absence of negative states.

A. Primary–Follower Architecture

Each shard consists of:

- One **Primary (Leader)** responsible for write serialization.
- Multiple **Followers** that replicate the leader's Write-Ahead Log (WAL).

All writes are processed through the leader, simplifying consistency enforcement while enabling horizontal scalability across shards.

Algorithm 4 Leader Promotion Procedure

Require: Followers F

```
Select follower with highest log index
Promote to Primary
Replay committed WAL entries
Resume write processing
```

B. Quorum-Based Commit Rule

A write is considered committed only after:

1) The leader appends the entry to its WAL and fsyncs.
2) The entry is replicated to followers.
3) A majority of replicas acknowledge persistence.

Let N denote the total replicas and $Q = \lceil N/2 \rceil$ the quorum size. A transaction commits only if at least Q replicas persist the entry.

Because any two majorities intersect, a newly elected leader must contain all previously committed entries. This quorum intersection property guarantees durability under leader failure.

C. Leader Election and Failover

Replicas exchange periodic heartbeats. If the primary misses consecutive heartbeats, it is declared failed and leader election is initiated.

Leader selection is based on log freshness rather than compute capacity. The follower with the highest replicated log index is promoted to ensure that no committed entries are lost.

D. Crash and Failure Scenarios

Crash Before Quorum: If the leader fails before majority acknowledgment, the entry is uncommitted and discarded during recovery.

Crash After Quorum: If majority acknowledgment has occurred, the entry exists on a quorum and survives leader promotion.

Follower Failure: Writes continue as long as quorum is maintained. Failed replicas recover by synchronizing missing WAL entries.

VII. Durability via Write-Ahead Logging

Durability is a fundamental requirement for financial transaction systems. Once a transaction is committed, it must survive process crashes, leader failures, and system restarts. The ledger achieves durability through a strictly enforced Write-Ahead Logging (WAL) mechanism.

A. Log-Before-Apply Principle

The system follows the *log-before-apply* rule:

> No state modification is applied to the ledger unless the corresponding log record has been durably persisted.

This ordering guarantees that committed state can always be reconstructed solely from log records.

Algorithm 5 Shard Crash Recovery

```
Read WAL sequentially from stable storage
for all log entries in order do
    if entry marked COMMITTED then
        Apply operation to ledger state
    else if entry marked PREPARED then
        Retain for coordinator reconciliation
    end if
end for
Reconstruct in-memory ledger state
```

B. WAL Structure

Each WAL entry contains:

- Transaction identifier (txnID)
- Operation type (DEBIT, CREDIT, PREPARED, COMMITTED, ABORTED)
- Account identifiers and transfer amount
- Timestamp

Entries are appended sequentially, forming an ordered history of shard-level operations.

C. Persistence Semantics

For each log append:

1) The entry is written to the WAL.
2) The WAL is synchronously flushed to stable storage (fsync).
3) The entry is replicated to followers.

State changes are applied only after durable persistence and quorum confirmation (as defined in the replication mechanism).

D. Crash Recovery Model

Upon shard restart, recovery proceeds as shown in Algorithm 5.

Committed entries are replayed deterministically. Uncommitted entries are ignored. Prepared entries remain pending until a final commit or abort decision is received.

E. Crash Scenarios

Crash Before Flush: If a crash occurs before fsync, the entry is absent from stable storage and no state modification has occurred.

Crash After Flush but Before Commit: The entry exists but remains uncommitted and is ignored during recovery.

Crash After Commit: Committed entries are reconstructed through WAL replay, ensuring that durable state is preserved.

F. Cross-Shard Durability

For cross-shard transactions, PREPARED and COMMITTED states are durably logged before acknowledgment. If a crash occurs between phases, the transaction remains in a recoverable state and can be safely resolved through coordinator replay. Idempotent handling prevents duplicate application.

G. Durability Guarantees

The WAL mechanism ensures:

- Committed transactions survive crashes and restarts.
- No partial ledger updates are applied.
- Ledger state after recovery reflects exactly the set of committed transactions.

VIII. Load Monitoring and Partition Rebalancing

To sustain high concurrency and prevent localized bottlenecks, the ledger incorporates a load monitoring and controlled partition rebalancing mechanism. The objective is to distribute workload evenly across shards while preserving transactional consistency and financial invariants.

A. Load Monitoring Architecture

Each shard periodically reports operational metrics to the Shard Load Monitor (SLM). These metrics include:

- Transaction throughput
- Request queue depth
- CPU and memory utilization
- Replication lag

Heartbeat messages serve two purposes:

- Detect node health and availability
- Provide real-time load visibility

The Load Monitor continuously evaluates shard metrics to detect sustained overload conditions.

B. Overload Detection Policy

A shard is classified as overloaded if one or more of the following conditions persist beyond a defined threshold:

- Throughput significantly exceeds cluster average.
- Request latency exceeds predefined percentile bounds.
- Resource utilization approaches saturation.

Detection is conservative to avoid oscillatory migrations. Only sustained imbalance triggers rebalancing.

C. Partition-Based Rebalancing Model

Data is organized into logical partitions determined by deterministic account hashing. Rebalancing occurs at the partition level rather than at the individual account level.

Let P denote a logical partition currently mapped to shard S_1.

When overload is detected:

1) The Load Monitor selects candidate partition P for migration.
2) The Load Monitor notifies the Transaction Coordinator to initiate migration.

D. Controlled Migration Procedure

To prevent inconsistent writes during migration, the system performs explicit partition halting and resumption.

Halt Phase: Before migration begins, the Coordinator temporarily halts new transactions targeting partition P. This ensures:

- No concurrent writes occur during transfer.
- WAL state is stable and fully replicated.

Algorithm 6 Partition Migration Procedure

Require: Partition P, source shard S_1, target shard S_2

Coordinator halts writes for P
Flush pending WAL entries for P on S_1
Transfer partition data from S_1 to S_2
Verify data integrity on S_2
Update Shard Map to map $P \rightarrow S_2$
Resume writes for P

Data Transfer Phase: The source shard transfers:

- Current ledger state for partition P
- Associated WAL entries necessary for consistency

The target shard reconstructs partition state before activation.

Shard Map Update: After successful transfer, the Shard Map is updated atomically. Subsequent routing decisions by the Coordinator use the new mapping.

Resume Phase: Once the mapping is updated, halted transactions for partition P are resumed and routed to the new shard.

E. Consistency Guarantees During Migration

The migration protocol ensures:

- No transaction is partially applied across shards.
- No writes occur during partition transfer.
- Routing remains deterministic before and after migration.

Because the partition is halted prior to transfer and WAL entries are flushed, state divergence cannot occur.

F. Interaction with Replication

Partition migration occurs at the shard leader level. The destination shard establishes replication with its own follower replicas before accepting writes. Thus, quorum-based durability guarantees remain intact after migration.

G. Scalability Characteristics

Rebalancing operates at partition granularity, allowing:

- Incremental redistribution of load.
- Addition of new shards without global downtime.
- Gradual scaling proportional to workload growth.

Throughput increases as new shards are introduced and partitions are redistributed.

H. Limitations

While partition-level migration improves load distribution, the system cannot split a single highly contended account across multiple shards. If extreme contention is concentrated on one account, that account remains bound to a single partition.

This trade-off simplifies routing and preserves consistency but limits scalability for single-key hotspots.

I. Design Trade-offs

The rebalancing mechanism prioritizes consistency and simplicity:

- Explicit halting increases temporary latency for affected partitions.
- Conservative migration policies prevent instability.
- Partition-level granularity simplifies correctness reasoning.

These trade-offs ensure that load redistribution does not compromise financial safety or transactional guarantees.

IX. Preliminary Evaluation and Design Analysis

At the current stage, the system architecture has been fully specified but not yet deployed in a production environment. Therefore, this section presents an analytical evaluation of performance characteristics, failure behavior, scalability projections, and design trade-offs based on the architectural model described earlier.

A. Analytical Latency Model

Transaction latency in the proposed system is dominated by synchronous log persistence and replication overhead.

1) Single-Shard Latency: A single-shard transaction requires:

- One WAL append
- One synchronous disk flush (fsync)
- One replication round to followers
- One quorum acknowledgment

Let:

$$T_{fsync} = \text{disk flush time}$$

$$T_{net} = \text{replication round-trip latency}$$

The approximate single-shard latency can be modeled as:

$$L_{single} \approx T_{fsync} + T_{net}$$

Because WAL writes are sequential and replication is parallelized to followers, the latency remains bounded primarily by storage persistence and network delay.

2) Cross-Shard Latency: Cross-shard transactions incur additional coordination overhead due to two-phase commit (2PC). Each participating shard performs:

- PREPARE WAL append + replication
- COMMIT WAL append + replication

Additionally, the coordinator introduces at least one extra messaging round.

The approximate latency becomes:

$$L_{cross} \approx 2(T_{fsync} + T_{net}) + T_{coord}$$

where T_{coord} represents coordinator communication delay.

Thus, cross-shard transactions are expected to exhibit roughly twice the persistence cost of single-shard transactions.

B. Throughput Scalability Projection

Because account data is deterministically partitioned, each shard operates independently for single-shard transactions.

If each shard sustains a minimum throughput of 100 transactions per second (TPS), the projected aggregate throughput becomes:

$$Throughput_{total} = N_{shards} \times 100$$

Number of Shards	Projected Throughput (TPS)
1	100
4	400
8	800
16	1600

This projection assumes a majority of transactions are single-shard. Workloads dominated by cross-shard transfers would experience additional coordination overhead, reducing effective throughput.

C. Consistency Under Cross-Shard Failures

During cross-shard transactions:

- PREPARED entries are durably logged before acknowledgment.
- COMMIT decisions are applied only after all shards confirm PREPARED.
- ABORT decisions revert prepared state consistently.

Even if a coordinator fails between phases, replayed messages are handled idempotently, preventing duplicate execution.

Thus, atomicity across shards is preserved despite partial failures.

D. Rebalancing Impact Analysis

Partition migration introduces temporary write suspension for affected partitions. While this increases latency for that subset of accounts, it prevents inconsistent routing and partial updates.

The system cannot split a single highly contended account across multiple shards. Therefore, extreme single-key hotspots remain a scalability limitation.

E. Trade-Off Analysis

The design prioritizes correctness and durability over minimal latency.

- Synchronous fsync increases write latency but guarantees crash consistency.
- Majority replication increases fault tolerance but adds network overhead.
- Two-phase commit ensures atomicity but introduces coordination delay.
- Partition-level rebalancing simplifies reasoning but limits hotspot mitigation.

These trade-offs are deliberate, reflecting the requirements of financial systems where correctness outweighs raw throughput.

F. Current Limitations and Future Evaluation

The present evaluation is analytical. Empirical benchmarking is planned to measure:

- p99 latency under varying shard counts.
- Throughput scaling under concurrent clients.
- Recovery time after primary failure.
- Overhead of synchronous fsync on SSD vs HDD.

Future work will validate projected performance metrics and refine migration heuristics based on observed workload characteristics.

X. Conclusion

This paper presented the design of a high-concurrency distributed ledger service based on deterministic sharding, quorum-based replication, write-ahead logging, and two-phase commit coordination. The layered architecture enables horizontal scalability while preserving strict financial correctness guarantees.

The transaction model ensures atomicity for both single-shard and cross-shard transfers, while majority-based replication and log-before-apply semantics provide durability under crash and failover scenarios. Controlled partition migration supports dynamic load redistribution without compromising consistency.

Analytical evaluation demonstrated predictable latency characteristics and linear throughput scaling with additional shards. Failure analysis confirmed preservation of ledger invariants under primary, replication, and coordination failures.

The design prioritizes correctness, durability, and recoverability over aggressive performance optimization, reflecting the requirements of financial systems. Future work includes empirical benchmarking, replication optimizations, improved migration strategies, and evaluation under high-contention workloads.

AI Usage Declaration

AI-assisted tools were used for language refinement, structural organization, and formatting guidance in preparing this report. All architectural decisions, system design, algorithms, analytical evaluation, and technical reasoning were independently developed by the authors. The use of AI tools did not replace conceptual understanding or original system design work.

References

[1] Cockroach Labs, "Architecture overview – CockroachDB," Technical Documentation, Jan. 2023. [Online]. Available: https://www.cockroachlabs.com/docs/stable/architecture/overview

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[3] FoundationDB Project, "A distributed unbundled transactional key-value store," Technical Report, 2021. [Online]. Available: https://www.foundationdb.org/files/fdb-paper.pdf

[4] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1992.

[5] C. Mohan, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.

[6] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, Stevenson, WA, USA, 2007.
[7] F. Chang *et al.*, "Bigtable: A distributed storage system for structured data," in *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Seattle, WA, USA, 2006.
[8] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Berkeley, CA, USA, 2004.
[9] H. Berenson *et al.*, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data (SIGMOD)*, San Jose, CA, USA, 1995.
[10] M. Stonebraker *et al.*, "H-Store: A high-performance, distributed main-memory transaction processing system," *Proc. VLDB Endowment*, vol. 1, no. 2, pp. 1496–1499, 2008.
[11] D. J. DeWitt and J. Gray, "Parallel database systems: The future of high-performance database systems," *Commun. ACM*, vol. 36, no. 6, pp. 85–98, Jun. 1992.

XI. Appendix

A. System Diagrams

This appendix presents the complete set of architectural and behavioral diagrams for the high-concurrency distributed ledger service.

1) Behavioral Diagrams:

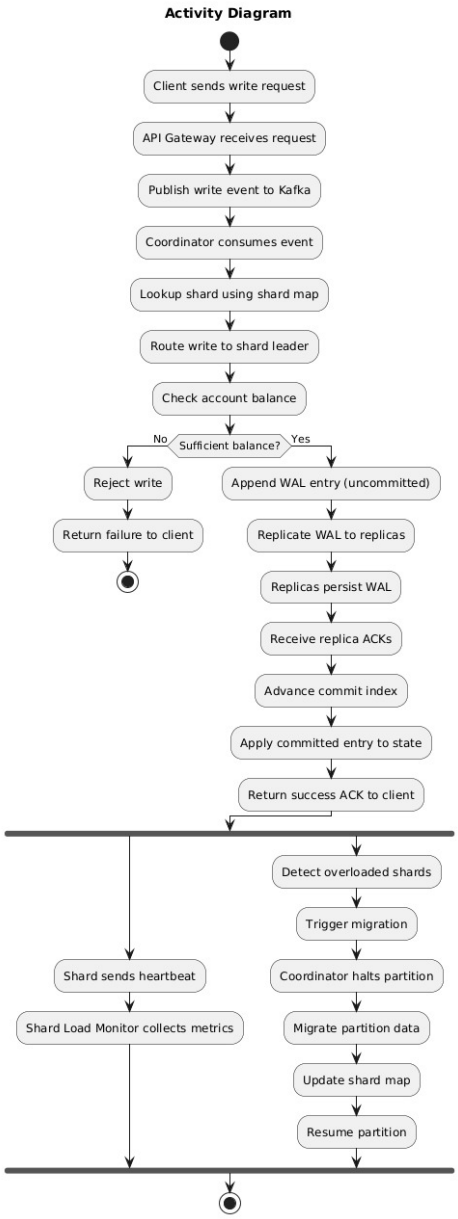

Fig. 1. Activity Diagram: Write Request Processing Flow

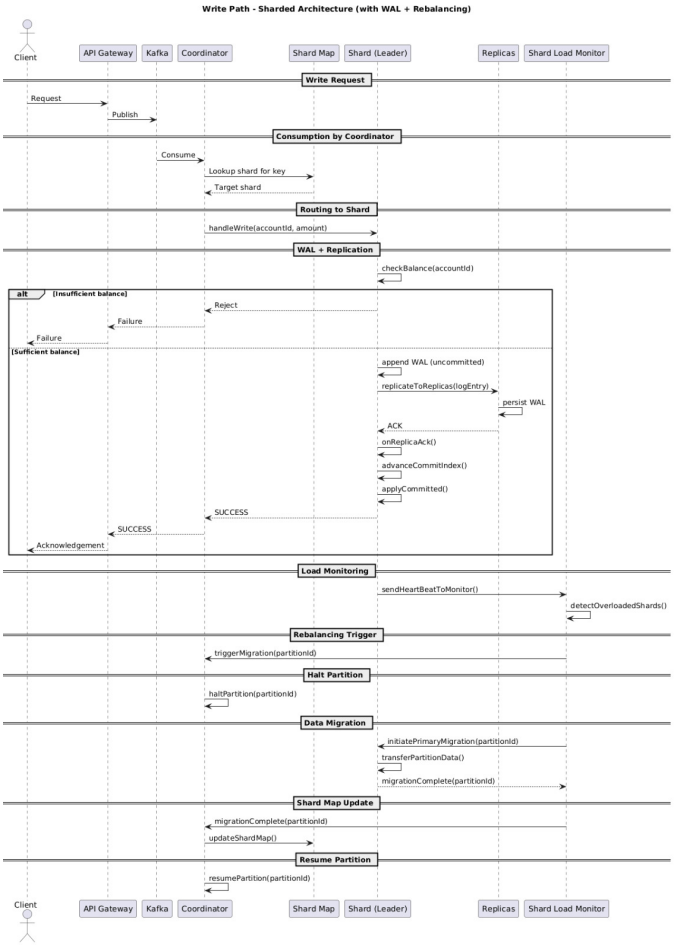

Fig. 2. Sequence Diagram: Sharded Architecture with Write-Ahead Log and Rebalancing

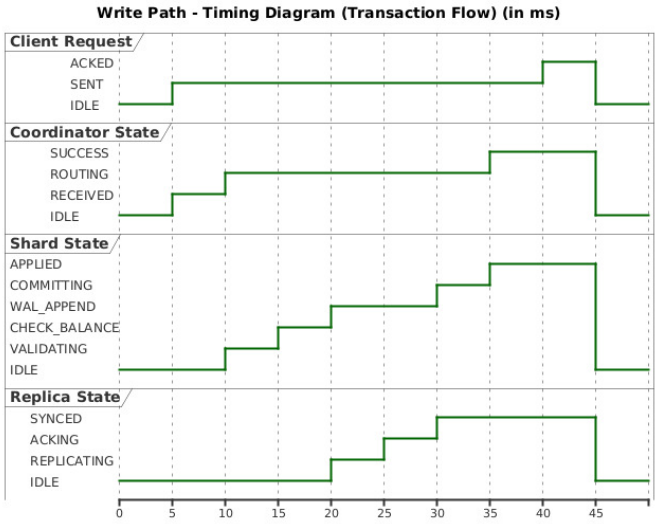

Fig. 3. Timing Diagram: Write Path with WAL Replication and Quorum Acknowledgment

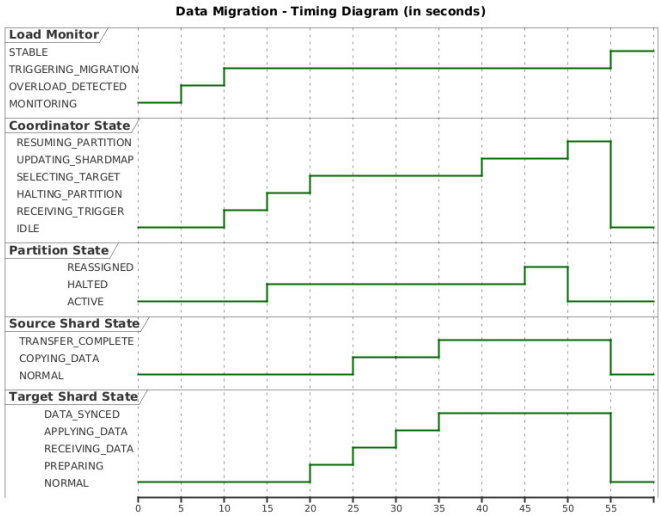

Fig. 4. Timing Diagram: Partition Migration and Shard Rebalancing

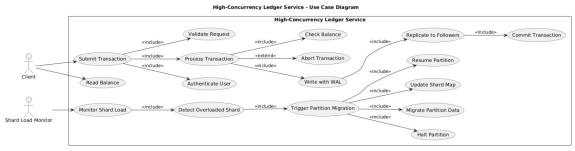

Fig. 5. Use Case Diagram: System Actor Interactions

2) Structural Diagrams:

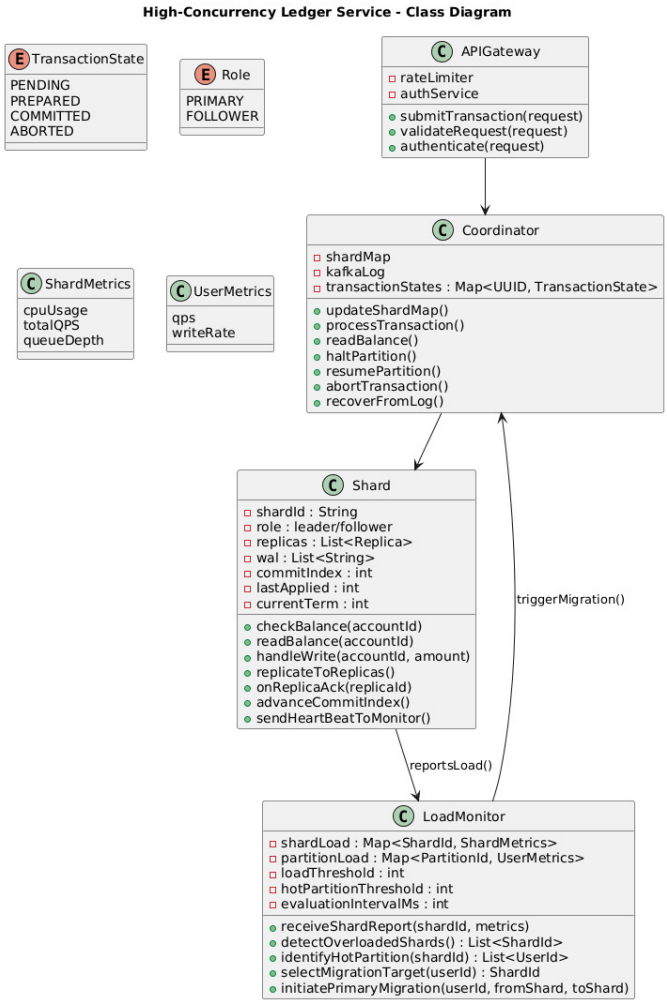

Fig. 6. Class Diagram: High-Concurrency Ledger Service Components

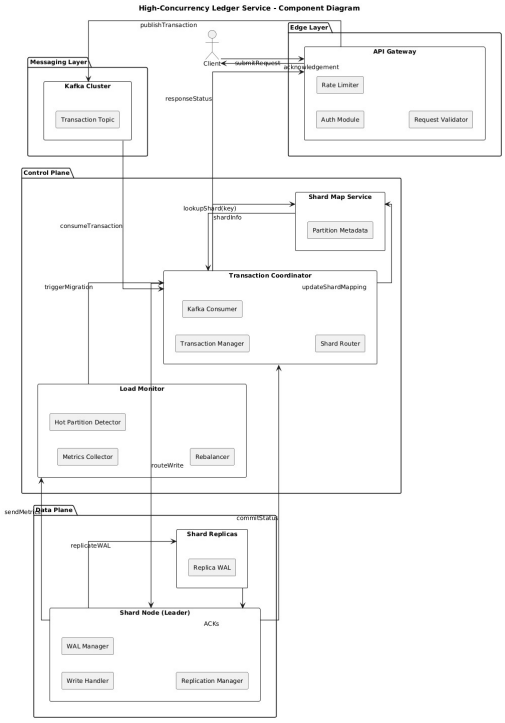

Fig. 7. Component Diagram: System Architecture and Inter-Component Dependencies

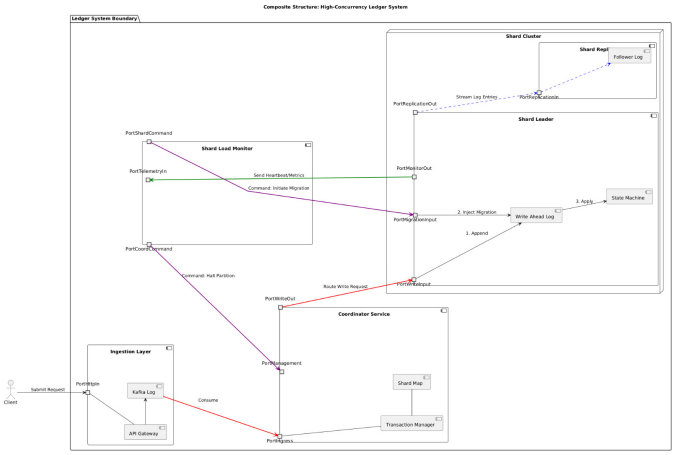

Fig. 8. Composite Structure Diagram: Internal Shard Organization and Replication